Homework 2 – Solutions

Security and Cryptography · MC1003 · With step-by-step explanations

Exercise 1 – One-time pad

Given. The intercepted ciphertext is

```
ct = 3b30cf656ed9fe4df54e08bc57e8f247c654977b7decfa9f2c171b
```

Mallory knows that the original message *m* starts with `Trust Charlie` and wants it to start with `Trust Mallory` (*m*'). Both strings are 13 bytes long. With the one-time pad, ct = m ⊕ k, where the key k is unknown to Mallory.

Idea

Mallory does not need the key. If she combines ct with the **difference** Δ = m ⊕ m' (which she can compute, since she knows both strings), the old message cancels out and the new one comes in:

```
ct' = ct ⊕ Δ = (m ⊕ k) ⊕ (m ⊕ m') = (m ⊕ m) ⊕ m' ⊕ k = 0 ⊕ m' ⊕ k = m' ⊕ k
```

This is exactly what encrypting m' under the same key k would give. When Bob decrypts:

```
ct' ⊕ k = (m' ⊕ k) ⊕ k = m'
```

Computation

Step 1. Difference Δ = m ⊕ m', byte by byte (the first 6 bytes, `Trust` and the space, are equal, so they give 00):

Position	**7**	**8**	**9**	**10**	**11**	**12**	**13**
m	C	h	a	r	l	i	e
m'	M	a	l	l	o	r	y
Δ = m ⊕ m'	0e	09	0d	1e	03	1b	1c

Hence Δ = 00 00 00 00 00 00 0e 09 0d 1e 03 1b 1c (13 bytes).

Step 2. Δ is applied only to the first 13 bytes of ct. The other 14 bytes are left untouched, because the statement only requires the message to *start* with Trust Mallory.

	First 13 bytes	**Rest (14 bytes)**
ct	3b 30 cf 65 6e d9 fe 4d f5 4e 08 bc 57	e8 f2 47 c6 54 97 7b 7d ec fa 9f 2c 17 1b
Δ	00 00 00 00 00 00 0e 09 0d 1e 03 1b 1c	(unchanged)
ct'	3b 30 cf 65 6e d9 f0 44 f8 50 0b a7 4b	e8 f2 47 c6 54 97 7b 7d ec fa 9f 2c 17 1b

Answer:

```
ct' = 3b30cf656ed9f044f8500ba74be8f247c654977b7decfa9f2c171b
```

Check and conclusion

$ct \oplus ct' = \Delta \parallel 0^{14}$: it gives Δ in the first 13 bytes and zeros in the remaining 14, as expected. This shows that the one-time pad is **malleable**: it provides perfect confidentiality but **no integrity**. To prevent this attack, the ciphertext must be authenticated with a MAC, or an AEAD mode (AES-GCM, ChaCha20-Poly1305) must be used.

Exercise 2 – 3-bit LFSR

At each step the new bit is $s_{n+3} = \oplus_{i \in T} s_{n+i}$: the XOR of the register positions (s_n, s_{n+1}, s_{n+2}) that belong to T. The register shifts and the output is s_0, s_1, s_2, ..., which starts with the initial state.

(a) T = {0}, initial state 0 1 0

With T = {0} only position 0 takes part, so $s_{n+3} = s_n$ (the bit three positions back is copied).

n	**(s_n, s_{n+1}, s_{n+2})**	**New bit $s_{n+3} = s_n$**
0	(0, 1, 0)	$s_3 = s_0 = 0$
1	(1, 0, 0)	$s_4 = s_1 = 1$
2	(0, 0, 1)	$s_5 = s_2 = 0$
3	(0, 1, 0)	$s_6 = s_3 = 0$
4	(1, 0, 0)	$s_7 = s_4 = 1$
5	(0, 0, 1)	$s_8 = s_5 = 0$

Answer (first 12 output bits):

```
0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, …
```

The sequence repeats every 3 bits (period 3). It does not reach the maximum period $2^3 - 1 = 7$, because this LFSR only copies one bit and does not mix.

(b) Output 1, 1, 1, 0, 0, 1, 0, 1, …: find T

The first three bits are the initial state (s_0 s_1 s_2 = 1 1 1). Each following bit gives an equation: it is the XOR of the positions in T. Rows n = 2, 3, 4 each isolate a single position, because only one register bit equals 1:

n	**(s_n, s_{n+1}, s_{n+2})**	**New bit**	**What follows**
2	(1, 0, 0)	$s_5 = 1$	Only position 0 equals 1 and the result is 1 ⇒ **$0 \in T$**
3	(0, 0, 1)	$s_6 = 0$	Only position 2 equals 1 and the result is 0 ⇒ **$2 \notin T$**
4	(0, 1, 0)	$s_7 = 1$	Only position 1 equals 1 and the result is 1 ⇒ **$1 \in T$**

Hence T = {0, 1}, i.e. $s_{n+3} = s_n \oplus s_{n+1}$. Each position was determined by a different row, so T is **unique**. Check against all rows:

n	(s_n, s_{n+1})	$s_n \oplus s_{n+1}$	actual s_{n+3}	
0	(1, 1)	0	0	✓
1	(1, 1)	0	0	✓
2	(1, 0)	1	1	✓
3	(0, 0)	0	0	✓
4	(0, 1)	1	1	✓

Answer: T = {0, 1}.

Exercise 3 – Linear complexity

According to the textbook (Definition 12.3), L(s) = 0 if s is the zero sequence; otherwise L(s) is the length of the shortest LFSR that generates s. To compute L we try L = 1, 2, 3, …: for each candidate we ask whether a rule $s_{n+L} = c_{L-1}s_{n+L-1} \oplus \ldots \oplus c_0 s_n$ (coefficients 0 or 1) holds for **all** n. The first L that works is L(s).

We know that $L(s \oplus t) \leq L(s) + L(t)$. Part (a) looks for a case where this bound is very loose, and part (b) for a case where it is tight.

(a) L(s ⊕ t) < min(L(s), L(t))

Example: s = 0, 1, 0, 1, 0, 1, … and t = 1, 0, 1, 0, 1, 0, … (t is s shifted by one position). Then s ⊕ t = 1, 1, 1, 1, 1, 1, …

Sequence	L	Justification
s = 0,1,0,1,…	2	The rule $s_{n+2} = s_n$ generates it. L = 1 is not enough: $s_{n+1} = c \cdot s_n$ with n = 0 gives $s_1 = c \cdot 0 = 0$, but $s_1 = 1$.
t = 1,0,1,0,…	2	The same rule $t_{n+2} = t_n$ generates it. L = 1 is not enough: $t_1 = c \cdot t_0$ gives c = 0, hence $t_2 = c \cdot t_1 = 0$, but $t_2 = 1$.
s ⊕ t = 1,1,1,…	1	The rule $u_{n+1} = u_n$ generates it, and it is not the zero sequence, so L ≠ 0.

Conclusion: L(s ⊕ t) = 1 < 2 = min(L(s), L(t)). Combining a sequence with its shifted copy cancels the differences and leaves a constant sequence, the simplest possible one that is not zero.

Alternative (trivial) example: with s = t any non-zero sequence, s ⊕ t is the zero sequence and L = 0 < L(s) = L(t). It is valid, but the example above is more illustrative because the XOR does not vanish.

(b) L(s ⊕ t) = L(s) + L(t)

Example: s = 1, 1, 1, 1, 1, 1, … and t = 1, 1, 0, 1, 1, 0, … (pattern 110 repeated). Then s ⊕ t = 0, 0, 1, 0, 0, 1, … (pattern 001 repeated).

Sequence	L	Justification
s = 1,1,1,…	1	The rule $s_{n+1} = s_n$ generates it and s is not zero.
t = 1,1,0,1,1,0,…	2	The rule $t_{n+2} = t_{n+1} \oplus t_n$ generates it: the pair (t_n, t_{n+1}) cycles through (1,1) → (1,0) → (0,1) → (1,1), and the rule gives the next bit in each case. L = 1 is not enough: $t_1 = c \cdot t_0$ gives c = 1, hence $t_2 = c \cdot t_1 = 1$, but $t_2 = 0$.
u = s ⊕ t = 0,0,1,0,0,1,…	3	The rule $u_{n+3} = u_n$ (period 3) generates it, so L ≤ 3. L = 1 is not enough: $u_1 = c \cdot u_0 = 0$ is consistent, but then $u_2 = c \cdot u_1 = 0$ while $u_2 = 1$. L = 2 is not enough either: with n = 0, $u_2 = c_1 u_1 \oplus c_0 u_0 = 0$, but $u_2 = 1$.

Conclusion: L(s ⊕ t) = 3 = 1 + 2 = L(s) + L(t).

Why it works. The two sequences have independent structures: one follows a rule of length 1 and the other a rule of length 2 with period 3, and they share no part. XOR cancels nothing, so the complexity adds up fully. In (a) the opposite happened: the two sequences were the same one shifted, and shared their whole structure.